Registry: Docker Hub explorer

1. Product and API discovery

Registry supports a small developer workflow: search for an image, inspect its repository, find a tag, and copy a pull command. It has two responsive pages, explicit repository search and debounced tag search, shareable URL state, and ten curated popular repositories. The original assignment is preserved in the repository.

Investigation began with the assignment's Docker Hub references, candidate endpoints supplied for investigation, and the official API reference. Small unauthenticated requests verified search, pagination, public details, tags, empty responses and missing repositories. Evidence is in docs/api-investigation.md.

Endpoints used (host: https://hub.docker.com):

- GET /v2/search/repositories/ with query, page and page_size.
- GET /v2/namespaces/{namespace}/repositories/{repository}.
- GET /v2/namespaces/{namespace}/repositories/{repository}/tags with page, page_size and optional name.

Details and tags routes are documented. Search and the tag-name filter were not found in the reference reviewed, so support is an observed integration dependency. Public tags worked anonymously despite authentication being shown in the tag reference. Search pages returned different results; name=alpine narrowed nginx tags across multiple pages. Deliberately nonexistent search terms returned zero matches; a missing repository returned HTTP 404.

Identity. Official nginx results had an empty owner and an unqualified name; details required namespace library. Nonofficial nginx/nginx-ingress also had an empty owner, so namespace comes from the qualified name. These differences are normalized in one adapter. The app does not guess ambiguous identities or assume every image has a latest tag.

Popular images and logos. The showcase selects nginx, Redis, PostgreSQL, Node.js, Python, MySQL, MongoDB, Ubuntu, Debian and Apache httpd. It is labeled as curated, not a measured global ranking. Descriptions and timestamps come from live requests. Ten Devicon logos are stored locally at a recorded source commit, with the MIT license and trademark attribution. Other results and failed images use an accessible initials fallback. This avoids decorative API requests; full logo coverage remains a limitation.

Sizes. Multi-platform responses contain different sizes and unknown-platform entries. The UI uses only linux/amd64 image size, labels that policy, and shows Unavailable when absent. It does not sum architectures or substitute an unexplained full_size value. Binary units describe reported image bytes; cached layers and other platforms can change transfer size.

2. Implementation and product decisions

Architecture. React and strict TypeScript run through Vite. A small Express server exposes four read operations: popular repositories, search, details and tags. A client API module separates fetch from rendering; a Docker Hub adapter owns upstream access and normalization. Shared contracts contain only consumed fields. Zod checks external responses at runtime. There is one package and no monorepo framework.

Why a server. Tested responses lacked Access-Control-Allow-Origin for localhost. The browser calls the local server, which also serves the built UI in production, including deep links. Production does not depend on Vite's development proxy. No credentials or persistence are required.

Request behavior. React Router stores search and page in URLs and preserves the originating explore URL in detail links. Repository search is explicit; tags update after a 500 ms pause, with immediate submission available. Automatic updates replace browser history and reset pagination. TanStack Query separates state by query key, passes abort signals and deduplicates requests. Client data is fresh for one minute; inactive entries expire after five minutes. Automatic retries, focus and reconnect refetching are disabled.

Failure boundaries. Details and tags load independently, with loading, empty, error and retry states. The server accepts known operations, bounds and validates input, encodes parameters, fixes page sizes, times out requests after ten seconds, and emits safe errors. It never follows arbitrary next-page URLs. A 429 sets a process-wide Retry-After cooldown, capped at one hour; the UI disables retry during that period. Only the fixed showcase has a five-minute server cache with in-flight deduplication to prevent repeated ten-request fan-out.

Useful additions. Server-side tag search and pagination help with large repositories. Visible commands make copying transparent and remain selectable if clipboard permission fails. Success appears only after writing and clears after 2.5 seconds. A selected read-only field supports manual copying if clipboard access fails. Clear controls preserve input focus; pagination returns focus to the results. Official-image labels, owners, UTC dates, platform labels, focus styles and mobile tag layout reduce ambiguity and support keyboard use.

Deliberate limits. No accounts, database, favorites, analytics, security scoring or deployment. The app does not fetch all tags, infer stability from latest, render upstream HTML descriptions, or claim its showcase is a live ranking. A platform selector and full search-logo coverage were deferred to keep implementation focused and explainable.

3. Verification, lessons and next steps

Automated verification. Strict type checking, ESLint, unit/integration tests and production build passed. Deterministic tests cover identity, exact commands, size policy, missing data, validation, upstream failures, cooldown and caching. Chromium scenarios use the production UI with fixture APIs: ten cards, explicit search, pagination reset, direct links and refresh, return context, clipboard contents and failure, independent tag errors and recovery, concurrent searches, keyboard use, narrow layouts and automated axe accessibility scans. Mocks prove behavior, not live availability.

Live verification. A separate smoke test queried all ten showcase repositories, searched and paginated nginx, retrieved namespaced details, searched tags across pages, checked empty results and handled a missing repository. Browser checks against the local production server inspected real search results, official and namespaced details, tags and exact clipboard content. The verification log records final checks and limits.

Lessons. Empty owner fields made raw responses unsuitable as UI contracts; an adapter kept special cases out of components. Multi-platform sizes needed an explicit policy rather than arithmetic. A rapid-search test exposed focus loss from remounting the input on URL changes; preserving the input and testing focus fixed it. Local TLS initially failed with Node's default CA store; using OS trust for that environment resolved it without disabling validation or changing application code.

Remaining risks. Observed endpoints can change; validation makes incompatible responses recoverable but cannot preserve a removed API. Tags and pagination are mutable. Logos cover ten projects. Keyboard and responsive checks do not constitute a complete screen-reader audit or cross-browser certification. Development tooling has moderate transitive minimatch audit findings; production dependencies have a clean audit. No forced major override was used to hide the tooling findings.

With more time. Add a platform selector with honest size semantics, partial showcase recovery, broader verified logos, contract monitoring, and screen-reader/Firefox/Safari checks. Before internet hosting, add application-level request limiting and operational monitoring.

AI assistance. AI assisted investigation, code, tests, visual review and documentation. Generated output was checked with compiler/lint tools, fixture tests, real API calls and browser interactions. These checks do not replace the candidate's responsibility to understand the solution; no claim of manual line-by-line candidate review is made.

References: https://docs.docker.com/reference/api/hub/latest/ ; https://github.com/echo-ha/registry-challange ; https://github.com/devicons/devicon (pinned commit and license included).